

Customer Churn Prediction (Classification)

Problem Statement

Customer churn is a major concern for businesses, especially in industries like telecom, banking, and SaaS. Losing customers leads to revenue loss and increased acquisition costs.

In this project, the goal is to build a Machine Learning classification model that predicts whether a customer will leave the company (Churn: Yes/No) based on features

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
data = pd.read_csv("telecom_churn_data.csv")
data
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	
...	...	...	...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	
7040	4801-JJAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	

7043 rows x 21 columns

Double-click (or enter) to edit

Basic Checks

```
data.shape
(7043, 21)

data.head()
Show hidden output
```

```
print(data.columns)
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
data.tail()
```

[Show hidden output](#)

```
data.info()
data.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null  object
1   gender                7038 non-null  object
2   SeniorCitizen          7043 non-null  int64
3   Partner               7043 non-null  object
4   Dependents            7043 non-null  object
5   tenure                7043 non-null  int64
6   PhoneService          7043 non-null  object
7   MultipleLines         7043 non-null  object
8   InternetService       7043 non-null  object
9   OnlineSecurity        7043 non-null  object
10  OnlineBackup          7043 non-null  object
11  DeviceProtection      7043 non-null  object
12  TechSupport           7043 non-null  object
13  StreamingTV           7043 non-null  object
14  StreamingMovies       7043 non-null  object
15  Contract              7043 non-null  object
16  PaperlessBilling      7041 non-null  object
17  PaymentMethod         7043 non-null  object
18  MonthlyCharges        7041 non-null  float64
19  TotalCharges          7038 non-null  object
20  Churn                 7043 non-null  object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7041.000000
mean	0.162147	32.371149	64.749581
std	0.368612	24.559481	30.085731
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

```
print(data.dtypes)
```

```
customerID      object
gender          object
SeniorCitizen   int64
Partner         object
Dependents      object
tenure          int64
PhoneService    object
MultipleLines   object
InternetService object
OnlineSecurity  object
OnlineBackup    object
DeviceProtection object
TechSupport     object
StreamingTV     object
StreamingMovies object
Contract        object
PaperlessBilling object
PaymentMethod   object
MonthlyCharges  float64
TotalCharges    object
Churn           object
dtype: object
```

```
cat_cols = data.select_dtypes(include='object')
cat_cols
```

	customerID	gender	Partner	Dependents	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	D
0	7590-VHVEG	Female	Yes	No	No	No phone service	DSL	No	Yes	
1	5575-GNVDE	Male	No	No	Yes	No	DSL	Yes	No	
2	3668-QPYBK	Male	No	No	Yes	No	DSL	Yes	Yes	
3	7795-CFOCW	Male	No	No	No	No phone service	DSL	Yes	No	
4	9237-HQITU	Female	No	No	Yes	No	Fiber optic	No	No	
...	...	...	...	...	...	...	...	...	...	
7038	6840-RESVB	Male	Yes	Yes	Yes	Yes	DSL	Yes	No	
7039	2234-XADUH	Female	Yes	Yes	Yes	Yes	Fiber optic	No	Yes	
7040	4801-JJAZL	Female	Yes	Yes	No	No phone service	DSL	Yes	No	
7041	8361-LTMKD	Male	Yes	No	Yes	Yes	Fiber optic	No	No	
7042	3186-AJIEK	Male	No	No	Yes	No	Fiber optic	Yes	No	

7043 rows × 18 columns

```
num_col = data.select_dtypes(include=['int64','float64'])
num_col
```

	SeniorCitizen	tenure	MonthlyCharges
0	0	1	29.85
1	0	34	56.95
2	0	2	53.85
3	0	45	42.30
4	0	2	70.70
...	...	...	...
7038	0	24	84.80
7039	0	72	103.20
7040	0	11	29.60
7041	1	4	74.40
7042	0	66	105.65

7043 rows × 3 columns

```
for x in cat_cols.drop(columns='customerID'):
    print(x, '-->', data[x].unique())
```

```
gender --> ['Female' 'Male' nan]
Partner --> ['Yes' 'No']
Dependents --> ['No' 'Yes']
PhoneService --> ['No' 'Yes']
MultipleLines --> ['No phone service' 'No' 'Yes']
InternetService --> ['DSL' 'Fiber optic' 'No']
OnlineSecurity --> ['No' 'Yes' 'No internet service']
OnlineBackup --> ['Yes' 'No' 'No internet service']
DeviceProtection --> ['No' 'Yes' 'No internet service']
TechSupport --> ['No' 'Yes' 'No internet service']
StreamingTV --> ['No' 'Yes' 'No internet service']
StreamingMovies --> ['No' 'Yes' 'No internet service']
Contract --> ['Month-to-month' 'One year' 'Two year']
PaperlessBilling --> ['Yes' 'No' nan]
PaymentMethod --> ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
'Credit card (automatic)']
TotalCharges --> ['29.85' '1889.5' '108.15' ... '346.45' '306.6' '6844.5']
Churn --> ['No' 'Yes']
```

```
for x in cat_cols.drop(columns='customerID'):
    print(data[x].value_counts())
    print()
```

```
No internet service    1526
Name: count, dtype: int64
```

```
TechSupport
No                3473
Yes               2044
No internet service    1526
Name: count, dtype: int64
```

```
StreamingTV
No                2810
Yes               2707
No internet service    1526
Name: count, dtype: int64
```

```
StreamingMovies
No                2785
Yes               2732
No internet service    1526
Name: count, dtype: int64
```

```
Contract
Month-to-month    3875
Two year          1695
One year          1473
Name: count, dtype: int64
```

```
PaperlessBilling
Yes              4169
No               2872
Name: count, dtype: int64
```

```
PaymentMethod
Electronic check    2365
Mailed check        1612
Bank transfer (automatic)  1544
Credit card (automatic)  1522
Name: count, dtype: int64
```

```
TotalCharges
20.2              11
                11
19.75              9
19.65              8
19.9              8
                ..
6849.4            1
692.35            1
130.15            1
3211.9            1
2065.15           1
Name: count, Length: 6526, dtype: int64
```

```
Churn
No               5174
Yes              1869
Name: count, dtype: int64
```

▼ Handle Missing values

```
print(data.isnull().sum())
```

```
customerID    0
gender        5
SeniorCitizen 0
Partner       0
Dependents    0
tenure        0
PhoneService  0
MultipleLines 0
InternetService 0
OnlineSecurity 0
OnlineBackup  0
DeviceProtection 0
TechSupport   0
StreamingTV   0
StreamingMovies 0
Contract      0
PaperlessBilling 2
PaymentMethod 0
MonthlyCharges 2
TotalCharges  5
```

```
Churn          0
dtype: int64
```

```
data.duplicated().sum()
```

```
np.int64(0)
```

Gender

```
data.loc[:, 'gender'].dtypes
```

```
dtype('O')
```

```
data.loc[:, 'gender'].mode()[0]
```

```
'Male'
```

```
data.loc[:, 'gender']=data.loc[:, 'gender'].fillna(data.loc[:, 'gender'].mode()[0])
```

PaperlessBilling

```
data.loc[:, 'PaperlessBilling'].dtypes
```

```
dtype('O')
```

```
data.loc[:, 'PaperlessBilling'].mode()[0]
```

```
'Yes'
```

```
data.loc[:, 'PaperlessBilling']=data.loc[:, 'PaperlessBilling'].fillna(data.loc[:, 'PaperlessBilling'].mode()[0])
```

MonthlyCharges

```
data.loc[:, 'MonthlyCharges'].dtypes
```

```
dtype('float64')
```

```
plt.figure(figsize=(5,2))
sns.histplot(data,x='MonthlyCharges',kde=True)
plt.show()
```

[Show hidden output](#)

```
data.loc[:, 'MonthlyCharges'].skew()
```

```
np.float64(-0.22028439456532767)
```

```
data.loc[:, 'MonthlyCharges'].median()
```

```
70.35
```

```
data.loc[:, 'MonthlyCharges']=data.loc[:, 'MonthlyCharges'].fillna(data.loc[:, 'MonthlyCharges'].median())
```

TotalCharges

```
data.loc[:, 'TotalCharges'].dtypes
```

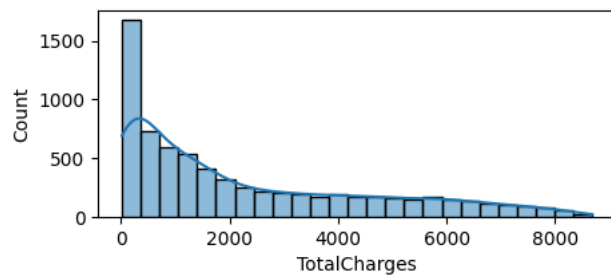
```
dtype('O')
```

```
data['TotalCharges'] = pd.to_numeric(data['TotalCharges'], errors='coerce')
```

```
data.loc[:, 'TotalCharges'].dtypes
```

```
dtype('float64')
```

```
plt.figure(figsize=(5,2))
sns.histplot(data,x='TotalCharges',kde=True)
plt.show()
```



```
data.loc[:, 'TotalCharges'].skew()
```

```
np.float64(0.9616058347613503)
```

```
data.loc[:, 'TotalCharges'].median()
```

```
1397.3
```

```
data.loc[:, 'TotalCharges']=data.loc[:, 'TotalCharges'].fillna(data.loc[:, 'TotalCharges'].median())
```

```
print(data.isnull().sum())
```

```
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

```
num_cols = data.select_dtypes(include=['int64', 'float64'])
num_cols
```

[Show hidden output](#)

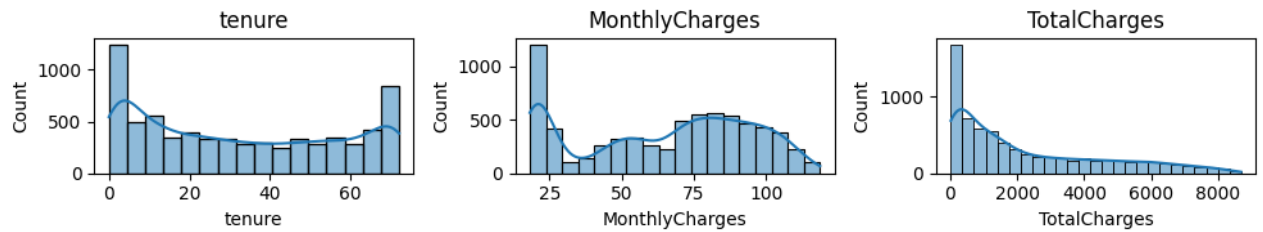
```
data.drop(columns={'customerID'}, inplace=True)
```

Univariate Analysis

- Distribution of numerical features
- Count plots for categorical features

```
num_cols1 = ['tenure', 'MonthlyCharges', 'TotalCharges']
plt.figure(figsize=(10,2))
for i, col in enumerate(num_cols1, 1):
    plt.subplot(1, 3, i) # 1 row, 3 columns
    sns.histplot(data[col], kde=True)
    plt.title(col)

plt.tight_layout()
plt.show()
```



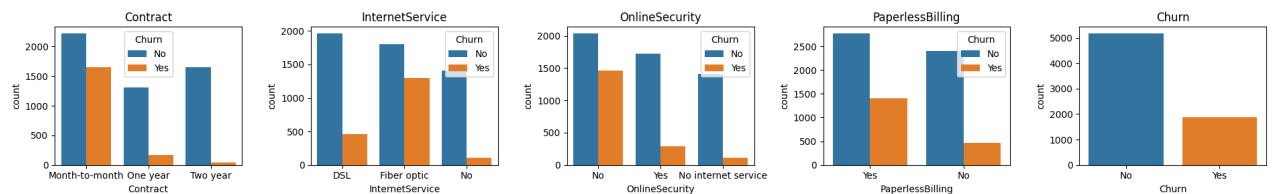
```
cat_cols1 = ['Contract', 'InternetService', 'OnlineSecurity', 'PaperlessBilling', 'Churn']
plt.figure(figsize=(19,3))
for i, col in enumerate(cat_cols1, 1):
    plt.subplot(1, 5, i) # 1 row, 5 columns
    sns.countplot(data=data, x=col)
    plt.title(col)

plt.tight_layout()
plt.show()
```

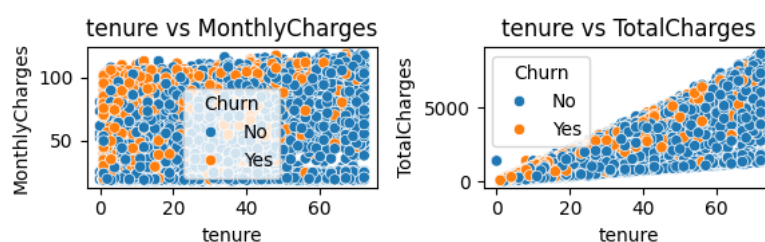


▼ Bivariate Analysis

```
cat_cols1 = ['Contract', 'InternetService', 'OnlineSecurity', 'PaperlessBilling', 'Churn']
plt.figure(figsize=(19,3))
for i, col in enumerate(cat_cols1, 1):
    plt.subplot(1, 5, i) # 1 row, 5 columns
    sns.countplot(data=data, x=col, hue='Churn')
    plt.title(col)
plt.tight_layout()
plt.show()
```



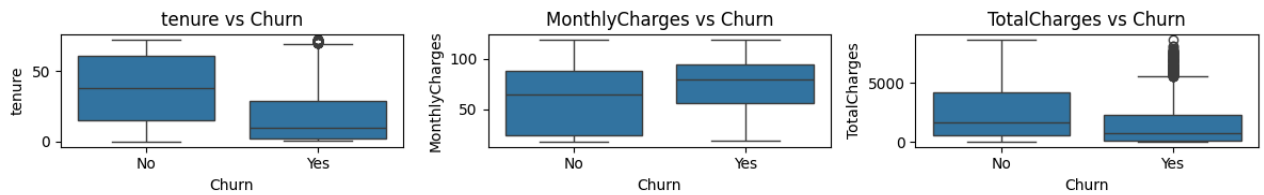
```
num_cols2 = ['MonthlyCharges', 'TotalCharges']
plt.figure(figsize=(6,2))
for i, col in enumerate(num_cols2, 1):
    plt.subplot(1, 2, i) # 1 row, 2 columns
    sns.scatterplot(data=data, x='tenure', y=col, hue='Churn')
    plt.title(f'tenure vs {col}')
plt.tight_layout()
plt.show()
```



```

num_cols1 = ['tenure', 'MonthlyCharges', 'TotalCharges']
plt.figure(figsize=(12,2))
for i, col in enumerate(num_cols1, 1):
    plt.subplot(1, 3, i)
    sns.boxplot(data=data, x='Churn', y=col)
    plt.title(f'{col} vs Churn')
plt.tight_layout()
plt.show()

```



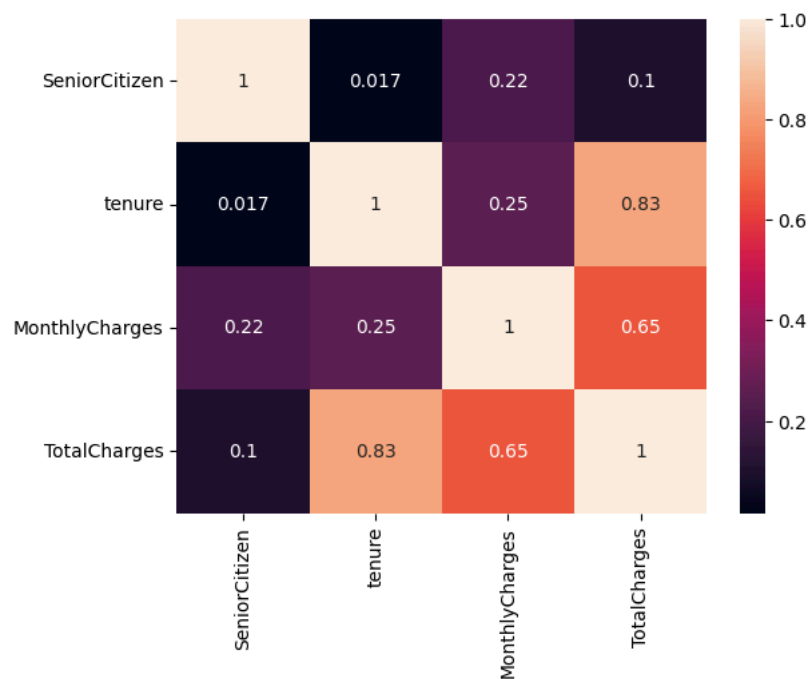
Correlation Heatmap

```
num_cols.corr()
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
SeniorCitizen	1.000000	0.016567	0.219901	0.102759
tenure	0.016567	1.000000	0.247712	0.825339
MonthlyCharges	0.219901	0.247712	1.000000	0.650435
TotalCharges	0.102759	0.825339	0.650435	1.000000

```
sns.heatmap(num_cols.corr(),annot=True)
```

<Axes: >



Handling Outliers

```

fig,ax = plt.subplots(1,3,figsize=(5,2))
sns.boxplot(data,x='tenure',ax=ax[0])
sns.boxplot(data,x='MonthlyCharges',ax=ax[1])
sns.boxplot(data,x='TotalCharges',ax=ax[2])
plt.tight_layout()

```

[Show hidden output](#)

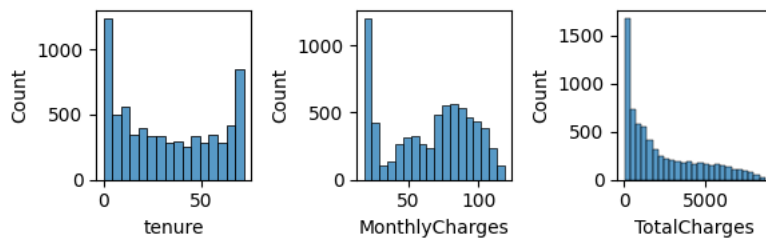
```

fig,ax = plt.subplots(1,3,figsize=(6,2))
sns.histplot(data,x='tenure',ax=ax[0])

```



```
sns.histplot(data,x='MonthlyCharges',ax=ax[1])
sns.histplot(data,x='TotalCharges',ax=ax[2])
plt.tight_layout()
```



```
A =['tenure', 'MonthlyCharges', 'TotalCharges']
for x in A:
    print(data[x].skew())
```

```
0.2395397495619829
-0.22046922813679604
0.9647294463449093
```

```
Q1 = data.loc[:, 'TotalCharges'].quantile(0.25)
Q3 = data.loc[:, 'TotalCharges'].quantile(0.75)
IQR = Q3-Q1
lower_limit= Q1 - 1.5 * IQR
upper_limit= Q3 + 1.5*IQR
```

```
# filter all the values greater than upper and less than lower
data[(data.loc[:, 'TotalCharges']>upper_limit)|(data.loc[:, 'TotalCharges']<lower_limit)]
```

[Show hidden output](#)

```
# %
len(data[(data.loc[:, 'TotalCharges']>upper_limit)|(data.loc[:, 'TotalCharges']<lower_limit)))/len(data)*100
```

```
0.0
```

```
mean_val = data.loc[:, 'tenure'].mean()
std_val = data.loc[:, 'tenure'].std()
lower_limit = mean_val - 3* std_val
upper_limit = mean_val + 3* std_val
```

```
# filter all the values greater than upper and less than lower
data[(data.loc[:, 'tenure']>upper_limit)|(data.loc[:, 'tenure']<lower_limit)]
```

```
gender SeniorCitizen Partner Dependents tenure PhoneService MultipleLines InternetService OnlineSecurity OnlineBa
```

```
# %
len(data[(data.loc[:, 'tenure']>upper_limit)|(data.loc[:, 'tenure']<lower_limit)))/len(data)*100
```

```
0.0
```

```
mean_val = data.loc[:, 'MonthlyCharges'].mean()
std_val = data.loc[:, 'MonthlyCharges'].std()
lower_limit = mean_val - 3* std_val
upper_limit = mean_val + 3* std_val
```

```
# filter all the values greater than upper and less than lower
data[(data.loc[:, 'MonthlyCharges']>upper_limit)|(data.loc[:, 'MonthlyCharges']<lower_limit)]
```

[Show hidden output](#)

```
# %
len(data[(data.loc[:, 'MonthlyCharges']>upper_limit)|(data.loc[:, 'MonthlyCharges']<lower_limit)))/len(data)*100
```

```
0.0
```

▼ Encoding

```
print(data.dtypes)
```

```

gender                object
SeniorCitizen         int64
Partner               object
Dependents            object
tenure                int64
PhoneService          object
MultipleLines         object
InternetService       object
OnlineSecurity        object
OnlineBackup          object
DeviceProtection      object
TechSupport           object
StreamingTV           object
StreamingMovies       object
Contract              object
PaperlessBilling      object
PaymentMethod         object
MonthlyCharges        float64
TotalCharges          float64
Churn                 object
dtype: object

```

```

for x in cat_cols.drop(columns='customerID'):
    print(x, '-->', data[x].unique())

```

```

gender --> ['Female' 'Male']
Partner --> ['Yes' 'No']
Dependents --> ['No' 'Yes']
PhoneService --> ['No' 'Yes']
MultipleLines --> ['No phone service' 'No' 'Yes']
InternetService --> ['DSL' 'Fiber optic' 'No']
OnlineSecurity --> ['No' 'Yes' 'No internet service']
OnlineBackup --> ['Yes' 'No' 'No internet service']
DeviceProtection --> ['No' 'Yes' 'No internet service']
TechSupport --> ['No' 'Yes' 'No internet service']
StreamingTV --> ['No' 'Yes' 'No internet service']
StreamingMovies --> ['No' 'Yes' 'No internet service']
Contract --> ['Month-to-month' 'One year' 'Two year']
PaperlessBilling --> ['Yes' 'No']
PaymentMethod --> ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
'Credit card (automatic)']
TotalCharges --> [ 29.85 1889.5  108.15 ...  346.45  306.6  6844.5 ]
Churn --> ['No' 'Yes']

```

```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
data.loc[:, 'partner'] = le.fit_transform(data.loc[:, 'Partner'])
data.loc[:, 'dependents'] = le.fit_transform(data.loc[:, 'Dependents'])
data.loc[:, 'Phone_Service'] = le.fit_transform(data.loc[:, 'PhoneService'])
data.loc[:, 'Paperless_Billing'] = le.fit_transform(data.loc[:, 'PaperlessBilling'])
data.loc[:, 'churn'] = le.fit_transform(data.loc[:, 'Churn'])

```

```

cols = ['MultipleLines', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']
for col in cols:
    data[col] = data[col].replace({
        'No internet service': 0,
        'No phone service': 0,
        'No': 1,
        'Yes': 2
    })

```

```

/tmp/ipykernel_2176/4246858082.py:3: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
data[col] = data[col].replace({

```

```

onehot_cols = ['gender', 'InternetService', 'Contract', 'PaymentMethod']
data = pd.get_dummies(data, columns=onehot_cols, dtype='int', drop_first=True)

```

```

data.drop(columns={'Churn', 'Partner', 'Dependents', 'PhoneService', 'PaperlessBilling'}, inplace=True)

```

```

print(data.dtypes)

```

[Show hidden output](#)

Scaling

```

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data[['tenure']] = scaler.fit_transform(data[['tenure']])

```

```
data[['MonthlyCharges']]=scaler.fit_transform(data[['MonthlyCharges']])
data[['TotalCharges']]=scaler.fit_transform(data[['TotalCharges']])
```

data

[Show hidden output](#)

Split data for x and y

```
x=data.drop(columns=['churn'])
y= data.loc[:, 'churn']
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

Balancing data

```
plt.figure(figsize=(5,2))
sns.countplot(data,x='churn')
plt.show()
```

[Show hidden output](#)

```
from imblearn.over_sampling import SMOTE
from collections import Counter
smote = SMOTE()
x_sampled,y_sampled = smote.fit_resample(x_train,y_train)
```

Counter(y_train)

Counter({0: 4138, 1: 1496})

Counter(y_sampled)

Counter({0: 4138, 1: 4138})

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression()
log_model.fit(x_sampled,y_sampled)
y_pred = log_model.predict(x_test)
```

```
from sklearn.metrics import confusion_matrix,accuracy_score,recall_score,precision_score,f1_score,roc_auc_score
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
[[794 242]
 [ 81 292]]
Accuracy:  0.7707594038325053
Recall:    0.7828418230563002
Precision: 0.5468164794007491
f1_score:  0.6438809261300992
```

KNN Classifier

```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier(n_neighbors=7)
knn_model.fit(x_sampled,y_sampled)
y_pred_1 = knn_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_1))
print("Accuracy: " ,accuracy_score(y_test,y_pred_1))
print('Recall: ',recall_score(y_test,y_pred_1))
```

```
print('Precision: ',precision_score(y_test,y_pred_1))
print('f1_score: ',f1_score(y_test,y_pred_1))
```

```
confusion matrix
[[710 326]
 [ 81 292]]
Accuracy: 0.7111426543647977
Recall: 0.7828418230563002
Precision: 0.47249190938511326
f1_score: 0.5893037336024218
```

▼ Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier()
dt_model.fit(x_sampled,y_sampled)
y_pred_2 = dt_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_2))
print("Accuracy: " ,accuracy_score(y_test,y_pred_2))
print('Recall: ',recall_score(y_test,y_pred_2))
print('Precision: ',precision_score(y_test,y_pred_2))
print('f1_score: ',f1_score(y_test,y_pred_2))
```

```
confusion matrix
[[794 242]
 [162 211]]
Accuracy: 0.7132718239886444
Recall: 0.5656836461126006
Precision: 0.46578366445916114
f1_score: 0.5108958837772397
```

```
from sklearn.tree import DecisionTreeClassifier
dt_tuned_model = DecisionTreeClassifier(criterion='gini',
                                       max_depth=50,max_features='sqrt',
                                       splitter='best',min_samples_leaf=5,
                                       min_samples_split=6,random_state=42)
dt_tuned_model.fit(x_sampled,y_sampled)
y_pred_2a = dt_tuned_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_2a))
print("Accuracy: " ,accuracy_score(y_test,y_pred_2a))
print('Recall: ',recall_score(y_test,y_pred_2a))
print('Precision: ',precision_score(y_test,y_pred_2a))
print('f1_score: ',f1_score(y_test,y_pred_2a))
```

```
confusion matrix
[[773 263]
 [124 249]]
Accuracy: 0.7253371185237757
Recall: 0.6675603217158177
Precision: 0.486328125
f1_score: 0.5627118644067797
```

▼ Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier()
rf_model.fit(x_sampled,y_sampled)
y_pred_3 = rf_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_3))
print("Accuracy: " ,accuracy_score(y_test,y_pred_3))
print('Recall: ',recall_score(y_test,y_pred_3))
print('Precision: ',precision_score(y_test,y_pred_3))
print('f1_score: ',f1_score(y_test,y_pred_3))
```

```
confusion matrix
[[854 182]
 [129 244]]
Accuracy: 0.7792760823278921
Recall: 0.6541554959785523
Precision: 0.5727699530516432
f1_score: 0.6107634543178974
```

```

from sklearn.ensemble import RandomForestClassifier
rf_tuned_model = RandomForestClassifier(n_estimators=10,max_depth=50,
                                      bootstrap=True,criterion='gini',
                                      max_features='sqrt',min_samples_split=6,
                                      min_samples_leaf=5,random_state=30)
rf_tuned_model.fit(x_sampled,y_sampled)
y_pred_3r = rf_tuned_model.predict(x_test)

```

```

print("confusion matrix\n", confusion_matrix(y_test,y_pred_3r))
print("Accuracy: " ,accuracy_score(y_test,y_pred_3r))
print('Recall: ',recall_score(y_test,y_pred_3r))
print('Precision: ',precision_score(y_test,y_pred_3r))
print('f1_score: ',f1_score(y_test,y_pred_3r))

```

```

confusion matrix
[[814 222]
 [ 89 284]]
Accuracy: 0.7792760823278921
Recall: 0.7613941018766756
Precision: 0.5612648221343873
f1_score: 0.646188850967008

```

▾ Gradient Boosting

```

from sklearn.ensemble import GradientBoostingClassifier
gb_model = GradientBoostingClassifier()
gb_model.fit(x_sampled,y_sampled)
y_pred_4 = gb_model.predict(x_test)

```

```

print("confusion matrix\n", confusion_matrix(y_test,y_pred_4))
print("Accuracy: " ,accuracy_score(y_test,y_pred_4))
print('Recall: ',recall_score(y_test,y_pred_4))
print('Precision: ',precision_score(y_test,y_pred_4))
print('f1_score: ',f1_score(y_test,y_pred_4))

```

```

confusion matrix
[[790 246]
 [ 78 295]]
Accuracy: 0.7700496806245565
Recall: 0.7908847184986595
Precision: 0.5452865064695009
f1_score: 0.6455142231947484

```

```

from sklearn.ensemble import GradientBoostingClassifier
gb_tuned_model = GradientBoostingClassifier(n_estimators=10,max_depth=50,
                                           learning_rate=0.01,
                                           max_features='sqrt',min_samples_split=6,
                                           min_samples_leaf=5)
gb_tuned_model.fit(x_sampled,y_sampled)
y_pred_4g = gb_tuned_model.predict(x_test)

```

```

print("confusion matrix\n", confusion_matrix(y_test,y_pred_4g))
print("Accuracy: " ,accuracy_score(y_test,y_pred_4g))
print('Recall: ',recall_score(y_test,y_pred_4g))
print('Precision: ',precision_score(y_test,y_pred_4g))
print('f1_score: ',f1_score(y_test,y_pred_4g))

```

```

confusion matrix
[[827 209]
 [104 269]]
Accuracy: 0.7778566359119943
Recall: 0.7211796246648794
Precision: 0.5627615062761506
f1_score: 0.6321974148061105

```

▾ XGBOOST Classifier

```

from xgboost import XGBClassifier
xgb_model = XGBClassifier()
xgb_model.fit(x_sampled,y_sampled)
y_pred_5 = xgb_model.predict(x_test)

```

```

print("confusion matrix\n", confusion_matrix(y_test,y_pred_5))
print("Accuracy: " ,accuracy_score(y_test,y_pred_5))
print('Recall: ',recall_score(y_test,y_pred_5))

```

```
print('Precision: ',precision_score(y_test,y_pred_5))
print('f1_score: ',f1_score(y_test,y_pred_5))
```

```
confusion matrix
[[844 192]
 [125 248]]
Accuracy: 0.7750177430801988
Recall: 0.6648793565683646
Precision: 0.5636363636363636
f1_score: 0.6100861008610086
```

```
from xgboost import XGBClassifier
xgb_tuned_model = XGBClassifier(n_estimators=50,max_depth=10,
                                learning_rate=0.1, gamma=0.1,subsample=0.8,
                                colsample_bytree=0.8,reg_alpha =0.1 ,
                                reg_lambda=1,n_jobs=-1)
xgb_tuned_model.fit(x_sampled,y_sampled)
y_pred_5x = xgb_tuned_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_5x))
print("Accuracy: " ,accuracy_score(y_test,y_pred_5x))
print('Recall: ',recall_score(y_test,y_pred_5x))
print('Precision: ',precision_score(y_test,y_pred_5x))
print('f1_score: ',f1_score(y_test,y_pred_5x))
```

```
confusion matrix
[[837 199]
 [109 264]]
Accuracy: 0.7814052519517388
Recall: 0.707774798927614
Precision: 0.5701943844492441
f1_score: 0.631578947368421
```

✧ Support Vector Classifier

```
from sklearn.svm import SVC
svc_model = SVC(probability=True) # Set probability=True for ROC AUC
svc_model.fit(x_sampled,y_sampled)
y_pred_6 = svc_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_6))
print("Accuracy: " ,accuracy_score(y_test,y_pred_6))
print('Recall: ',recall_score(y_test,y_pred_6))
print('Precision: ',precision_score(y_test,y_pred_6))
print('f1_score: ',f1_score(y_test,y_pred_6))
```

```
confusion matrix
[[799 237]
 [ 89 284]]
Accuracy: 0.7686302342086586
Recall: 0.7613941018766756
Precision: 0.54510556621881
f1_score: 0.6353467561521253
```

```
from sklearn.svm import SVC
svc_tuned_model = SVC(kernel = 'rbf',gamma=0.01,C=20, probability=True)
svc_tuned_model.fit(x_sampled,y_sampled)
y_pred_6s = svc_tuned_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_6s))
print("Accuracy: " ,accuracy_score(y_test,y_pred_6s))
print('Recall: ',recall_score(y_test,y_pred_6s))
print('Precision: ',precision_score(y_test,y_pred_6s))
print('f1_score: ',f1_score(y_test,y_pred_6s))
```

```
confusion matrix
[[796 240]
 [ 83 290]]
Accuracy: 0.7707594038325053
Recall: 0.7774798927613941
Precision: 0.5471698113207547
f1_score: 0.6423034330011074
```

```
y_prob_log = log_model.predict_proba(x_test)[: ,1]
y_prob_1 = knn_model.predict_proba(x_test)[: ,1]
y_prob_2a = dt_model.predict_proba(x_test)[: ,1]
y_prob_3r = rf_model.predict_proba(x_test)[: ,1]
y_prob_4g = gb_model.predict_proba(x_test)[: ,1]
```

```
y_prob_5x = xgb_model.predict_proba(x_test)[: ,1]
y_prob_6s = svc_tuned_model.predict_proba(x_test)[: ,1]
```

```
from sklearn.metrics import roc_auc_score
print("Logistic ROC:", roc_auc_score(y_test, y_prob_log))
print("Decision Tree (Tuned) ROC:", roc_auc_score(y_test, y_prob_2a))
print("Random Forest (Tuned) ROC:", roc_auc_score(y_test, y_prob_3r))
print("KNN ROC:", roc_auc_score(y_test, y_prob_1))
print("Gradient Boosting (Tuned) ROC:", roc_auc_score(y_test, y_prob_4g))
print("XGBoost (Tuned) ROC:", roc_auc_score(y_test, y_prob_5x))
print("SVC (Tuned) ROC:", roc_auc_score(y_test, y_prob_6s))
```

```
Logistic ROC: 0.8523593010858427
Decision Tree (Tuned) ROC: 0.6666558841491816
Random Forest (Tuned) ROC: 0.8306903226474274
KNN ROC: 0.798760700570352
Gradient Boosting (Tuned) ROC: 0.8534384154357345
XGBoost (Tuned) ROC: 0.8337102901446066
SVC (Tuned) ROC: 0.8469223244692413
```

Grid Search CV

```
from sklearn.model_selection import GridSearchCV
svc_gs_model = SVC(probability=True)
parameters = {'kernel': ['rbf'], 'gamma': [0.01, 0.1], 'C': [10]}
grid = GridSearchCV(estimator=svc_gs_model, param_grid=parameters, cv=2, scoring='accuracy', n_jobs=-1)
grid.fit(x_sampled, y_sampled)
svc_gs_model = grid.best_estimator_
```

```
print(grid.best_params_)
```

```
{'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}
```

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', gamma=0.01, C=30, probability=True)
model.fit(x_sampled, y_sampled)
y_pred_g = svc_gs_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test, y_pred_g))
print("Accuracy: ", accuracy_score(y_test, y_pred_g))
print('Recall: ', recall_score(y_test, y_pred_g))
print('Precision: ', precision_score(y_test, y_pred_g))
print('f1_score: ', f1_score(y_test, y_pred_g))
```

```
confusion matrix
[[805 231]
 [111 262]]
Accuracy: 0.7572746628814763
Recall: 0.7024128686327078
Precision: 0.5314401622718052
f1_score: 0.605080831408776
```

Randomised Search CV

```
from sklearn.model_selection import RandomizedSearchCV
svc_rs_model = SVC(probability=True)
parameters = {'kernel': ['rbf'],
              'gamma': [0.01, 0.1],
              'C': [10]}
random = RandomizedSearchCV(n_iter=5, estimator=svc_rs_model, cv=5,
                           param_distributions=parameters, scoring='accuracy', n_jobs=-1)
random.fit(x_sampled, y_sampled)
svc_rs_model = random.best_estimator_
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:317: UserWarning: The total space of parameters 2
warnings.warn(
```

```
print(random.best_params_)
```

```
{'kernel': 'rbf', 'gamma': 0.1, 'C': 10}
```

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', gamma=0.5, C=5, probability=True)
```

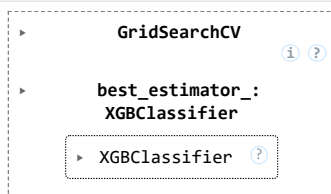
```
model.fit(x_sampled,y_sampled)
y_pred_r = svc_rs_model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_r))
print("Accuracy: " ,accuracy_score(y_test,y_pred_r))
print('Recall: ',recall_score(y_test,y_pred_r))
print('Precision: ',precision_score(y_test,y_pred_r))
print('f1_score: ',f1_score(y_test,y_pred_r))
```

```
confusion matrix
[[805 231]
 [111 262]]
Accuracy: 0.7572746628814763
Recall: 0.7024128686327078
Precision: 0.5314401622718052
f1_score: 0.605080831408776
```

Grid Search CV for XGboost

```
model = XGBClassifier()
parameters = {'max_depth':[3,5,7], 'n_estimators':[50,100],
              'learning_rate':[0.1,0.2], 'reg_alpha':[0.1],
              'reg_lambda':[0.1]}
grid = GridSearchCV(estimator=model,param_grid=parameters,cv= 3,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```



```
print(grid.best_params_)
```

```
{'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 100, 'reg_alpha': 0.1, 'reg_lambda': 0.1}
```

MLP Classifier

```
from sklearn.neural_network import MLPClassifier
mlp_model = MLPClassifier()
mlp_model.fit(x_sampled,y_sampled)
y_pred_m = mlp_model.predict(x_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_m))
print("Accuracy: " ,accuracy_score(y_test,y_pred_m))
print('Recall: ',recall_score(y_test,y_pred_m))
print('Precision: ',precision_score(y_test,y_pred_m))
print('f1_score: ',f1_score(y_test,y_pred_m))
```

```
confusion matrix
[[833 203]
 [119 254]]
Accuracy: 0.7714691270404542
Recall: 0.6809651474530831
Precision: 0.5557986870897156
f1_score: 0.6120481927710844
```

```
from sklearn.neural_network import MLPClassifier
mlp_tuned_model = MLPClassifier(hidden_layer_sizes=(20,20),activation='relu',solver='adam',random_state=42)
mlp_tuned_model.fit(x_sampled,y_sampled)
y_pred_hm = mlp_tuned_model.predict(x_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred_hm))
print("Accuracy: " ,accuracy_score(y_test,y_pred_hm))
print('Recall: ',recall_score(y_test,y_pred_hm))
```



```
print('Precision: ',precision_score(y_test,y_pred_hm))
print('f1_score: ',f1_score(y_test,y_pred_hm))
```

```
confusion matrix
[[722 314]
 [ 73 300]]
Accuracy: 0.7253371185237757
Recall: 0.8042895442359249
Precision: 0.48859934853420195
f1_score: 0.60790273556231
```

▼ Comparison Model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
```

```
models = {
    "Logistic": (y_pred, y_prob_log),
    "KNN": (y_pred_1, y_prob_1),
    "Decision Tree": (y_pred_2a, y_prob_2a),
    "Random Forest": (y_pred_3r, y_prob_3r),
    "Gradient Boosting": (y_pred_4g, y_prob_4g),
    "XGBoost": (y_pred_5x, y_prob_5x),
    "SVC": (y_pred_6s, y_prob_6s)
}
```

```
for name, (y_pred, y_prob) in models.items():
```

```
    acc = accuracy_score(y_test, y_pred)
    pre = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)
```

```
    print("MODEL:", name)
    print("Accuracy:", acc)
    print("Precision:", pre)
    print("Recall:", rec)
    print("F1 Score:", f1)
    print("ROC-AUC:", auc)
    print("-----")
```

```
MODEL: Logistic
Accuracy: 0.7707594038325053
Precision: 0.5468164794007491
Recall: 0.7828418230563002
F1 Score: 0.6438809261300992
ROC-AUC: 0.8523593010858427
-----
```

```
MODEL: KNN
Accuracy: 0.7111426543647977
Precision: 0.47249190938511326
Recall: 0.7828418230563002
F1 Score: 0.5893037336024218
ROC-AUC: 0.798760700570352
-----
```

```
MODEL: Decision Tree
Accuracy: 0.7253371185237757
Precision: 0.486328125
Recall: 0.6675603217158177
F1 Score: 0.5627118644067797
ROC-AUC: 0.6666558841491816
-----
```

```
MODEL: Random Forest
Accuracy: 0.7792760823278921
Precision: 0.5612648221343873
Recall: 0.7613941018766756
F1 Score: 0.646188850967008
ROC-AUC: 0.8306903226474274
-----
```

```
MODEL: Gradient Boosting
Accuracy: 0.7778566359119943
Precision: 0.5627615062761506
Recall: 0.7211796246648794
F1 Score: 0.6321974148061105
ROC-AUC: 0.8534384154357345
-----
```

```
MODEL: XGBoost
Accuracy: 0.7814052519517388
Precision: 0.5701943844492441
Recall: 0.707774798927614
F1 Score: 0.631578947368421
ROC-AUC: 0.8337102901446066
-----
```

```
MODEL: SVC
```

Accuracy: 0.7707594038325053
Precision: 0.5471698113207547
Recall: 0.7774798927613941
F1 Score: 0.6423034330011074
ROC-AUC: 0.8469223244692413

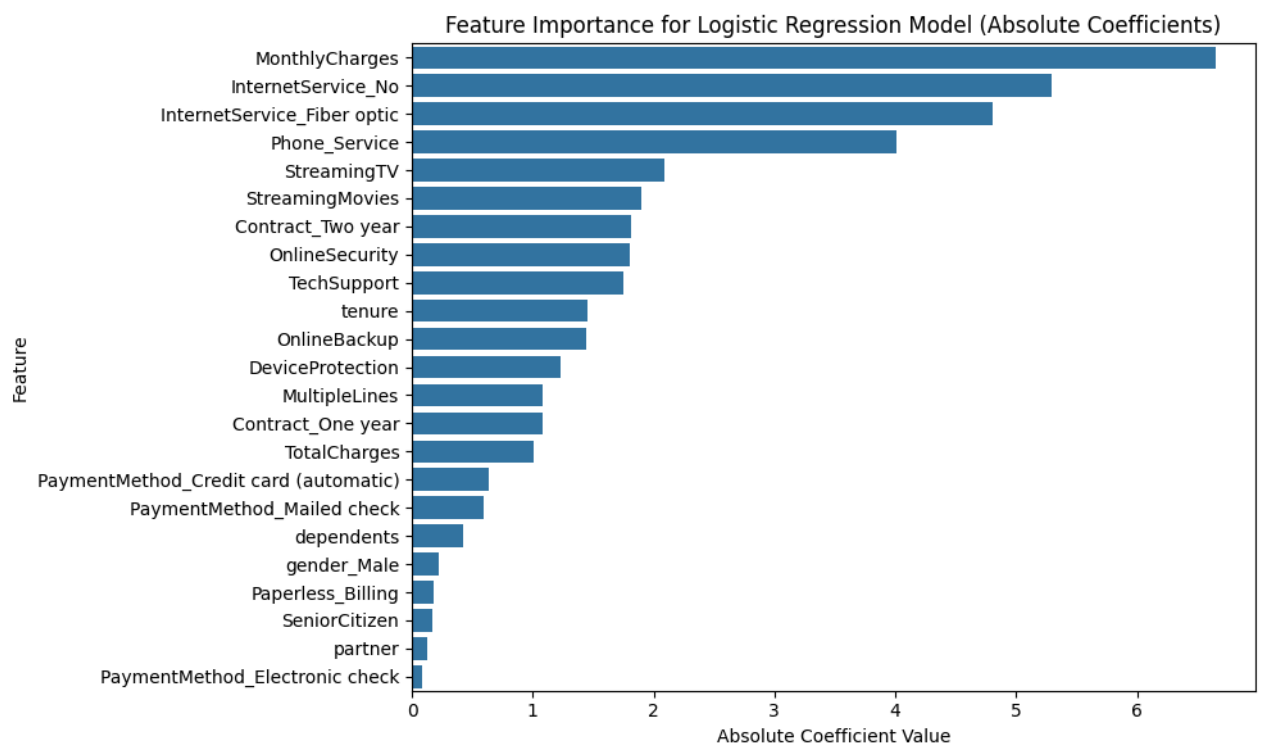
Based on the F1 Score, ROC-AUC and Recall Score, the Logistic Regression model performed the best with an F1 Score of 0.6439, ROC-AUC score of 0.8523 and Recall score of 0.78016. This indicates a good balance between precision and recall for predicting customer churn.

Feature Importance Plot (Logistic Regression Model)

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Get coefficients from the Logistic Regression model
# The coefficients represent the feature importances for linear models
feature_coefficients = log_model.coef_[0]
feature_names = x_sampled.columns
importance_df_lr = pd.DataFrame({
    'Feature': feature_names,
    'Importance': abs(feature_coefficients)
})

importance_df_lr = importance_df_lr.sort_values(by='Importance', ascending=False)
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df_lr)
plt.title('Feature Importance for Logistic Regression Model (Absolute Coefficients)')
plt.xlabel('Absolute Coefficient Value')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```



Monthly charges is the important feature in logistic regression model